

LPU Backend Case Studies 2026

Plus Points in Implementation (Overall Evaluation Criteria)

1. Authentication:

- Implement robust user authentication protocols to ensure secure access.

2. Cost Estimation - Time and Space:

- Conduct a thorough analysis of time and space complexity in the system.
- Utilize efficient algorithms and data structures to optimize both time and space requirements.

3. Handling System Failure Cases:

- Implement fault-tolerant mechanisms to address system failures.
- Employ backup and recovery strategies for data integrity.
- Develop comprehensive error recovery procedures to minimize downtime.

4. Object-Oriented Programming Language (OOPS):

- Choose a robust OOPS language for structured and modular code.
- Leverage OOPS principles such as encapsulation, inheritance, and polymorphism for maintainability and extensibility.

5. Trade-offs in the System:

- Clearly define and document trade-offs made during system design.
- Evaluate and communicate the rationale behind architectural and design decisions.
- Consider trade-offs in terms of performance, scalability, and maintainability.

6. System Monitoring:

- Implement comprehensive monitoring tools to track system performance.
- Utilize real-time dashboards and logging mechanisms to promptly identify and address issues.

7. Caching:

- Integrate caching mechanisms to enhance system response times.
- Utilize caching for frequently accessed data to reduce database load.
- Implement cache eviction policies for optimal resource utilization.

8. Error and Exception Handling:

- Develop a robust error and exception handling framework.

- Provide meaningful error messages for effective debugging.
 - Regularly review and update error-handling strategies based on system usage patterns.
-

Instructions

1. Read and Understand the Problem Statement:

- Carefully read the case study you pick. Understand the requirements, the inputs, the expected outputs, and any rules mentioned.

2. Choose a Programming Language:

- Select a language you are comfortable with. An object-oriented language (Java, C#, Python, etc.) is preferred.

3. Design Your Solution:

- Plan the structure first. Decide your database tables (ER diagram), the main classes, and the APIs you will build before writing code.

4. Write the Code:

- Write clean code with meaningful names and proper layering (controller / service / database).
- Break the problem into small functions and modules so it is easy to read and test.

5. Test Your Code:

- Test with normal inputs AND edge cases (empty data, wrong input, duplicate requests).
- Add a few unit tests for the important logic.

6. Document Your Code:

- Add a README: how to run the project, what assumptions you made, and a short note on the choices/trade-offs you took.

7. Submit Your Solution:

- Push your code to a public GitHub repository and share the link.
- Keep a meaningful commit history – do not upload everything in one commit.

8. Demonstration:

- Add a short demo video or screenshots showing the main features and a few sample API calls (Postman / curl).

5. Smart Desk Booking – Seat Your Team Together

In hybrid offices employees don't own a fixed desk – they book one for the days they come in, like booking a seat in a cinema. But a good system does more than hand out any free desk: it tries to seat teammates near each other, respects how many desks each team/floor may take, and – above all – never gives the same desk to two people. Doing this correctly when hundreds of people book at the same moment is the real test.

I. Booking a Desk

Employees book a desk on a floor for a date (and time window); they can cancel before a cut-off.

- ☒ **Book / Cancel** – Reserve a desk for a day, cancel before the cut-off, and see the assigned desk.
- ☒ **Desk Types** – Support fixed desks (reserved for a named person) and hot desks (a shared pool anyone on the team can grab).

II. No Double-Booking Under Concurrency (the hard part)

Two employees tapping “book” on the last free desk at the same instant must not both get it. This is a classic race condition.

- ☒ **Exactly One Winner** – Guarantee a desk-day can be held by only one person, even under simultaneous requests.
- ☒ **Explain Your Locking** – State whether you use optimistic or pessimistic locking (or a DB constraint) and why – and what the loser sees.

III. Team Neighbourhood Placement

When possible, place people from the same team close together on the floor, using the desks' positions on the map.

- ☒ **Sit Together** – Prefer desks within a short distance of a team's already-booked desks (use desk x/y coordinates).
- ☒ **Zone / Team Quotas** – Respect caps like “Team A $\leq$ 8 desks on Floor 3” and “Floor 3 max 60 people”.

Example Use Case

- 6 teammates book for Tuesday → the system clusters them in one corner of Floor 3 instead of scattering them across the building.

IV. No-Show Release & Fairness

A booked-but-empty desk is wasted – detect it and give it back to the pool.

- ☒ **Auto-Release** – If the employee hasn't checked in within a grace period, free the desk for others.
- ☒ **Cut-Off & Windows** – Enforce booking windows in the employee's own timezone (careful with cut-off boundaries).

Outcome

- Desks are used well, teams sit together, and nobody ever finds someone else at “their” desk.



Points to Take Care Of (this is what we look for):

- The double-booking race is the heart of this problem – prove with a test (two simultaneous bookings) that only one wins.
 - Neighbourhood placement is a small clustering/assignment problem – explain how you keep it fast for a 500-desk floor (don’t compare every desk pair each time).
 - Quotas and no-show release interact: a released desk must respect team/zone caps before being re-offered.
 - Timezone and cut-off edge cases matter – a booking exactly at the cut-off should behave predictably.
-

